

**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND
MANAGEMENT
A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE
UNIVERSITY)**

**BALMATTA, MANGALORE
Final Project Report**

on

**DESIGN AND DEVELOPMENT OF NATURAL LANGUAGE
INTERFACE FOR AGRICULTURE INFRASTRUCTURE**

Student Name: MOHAMMED SHAN S

Course : BCA Cloud Computing And Devops IBM&TCS

Reg No : 23BCAICD066

Project Code : PRJN26-115

Industry Mentor: Sumit Kumar Shukla

Guided by: MS Deepthi

CERTIFICATE

This is to certify that the project entitled

**“DESIGN AND DEVELOPMENT OF NATURAL LANGUAGE INTERFACE FOR
AGRICULTURE INFRASTRUCTURE”**

submitted by **MOHAMMED SHAN**, III BCA student of **Yenepoya Institute of Arts, Science, Commerce and Management, Mangalore**, is a bonafide record of the internship work carried out by him under our supervision and guidance, in partial fulfilment of the requirement for the award of the degree of Bachelor of Computer Application from Yenepoya (Deemed to be University), Mangalore, during the academic year 2025–2026.

Internal Guide :

Industry IBM Mentor:

MS Deepthi Faculty Guide Sumit Kumar Shukla Dept. of
Computer Science Industry Mentor,

IBM

HOD

DR. Rathnakar Shetty P
Dept. of Computer Science
Yenepoya Institute of Arts, Science, Commerce and
Management

DECLARATION

I hereby declare that the project entitled

“Design and Development of Natural Language Interface for Agriculture Infrastructure”

submitted to Yenepoya Institute of Arts, Science, Commerce and Management, Mangalore, in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications under Yenepoya (Deemed to be University), Mangalore, is an original work carried out by me under the guidance of Ms. Deepthi, Department of Computer Science.

I further declare that this project has not been submitted previously, either in full or in part, to any other institution for the award of any degree or diploma, and all the information presented in this report is true to the best of my knowledge.

**Place: Mangalore Date: May
2026**

**MOHAMMED SHAN
III Year BCA**

TABLE OF CONTENTS

Executive Summary	1
Background	2
o Aim	3
o Technologies	4
o Hardware Architecture	5
o Software Architecture	6
System	8
o Requirements	8
- Functional	8
- User	9
- Environmental	10
o Design and Architecture	11
o Implementation	13
o Testing	16
o GUI Layout	18
o Customer testing	21
o Evaluation	23
Snapshots of the Project	26
Conclusions	30
Further development or research	32
References	33
Appendix	34

EXECUTIVE SUMMARY

The Agri Assistant is a comprehensive, AI-powered agricultural advisory chatbot developed as part of an IBM internship project. The system provides farmers, agronomists, and agricultural students with real-time, knowledge-based guidance on crop management, pest and disease control, fertilizer recommendations, irrigation strategies, market pricing, and organic farming practices.

The project exists in three versions:

- Terminal Chatbot: Python-based, runs in VS Code command line.
- Website Chatbot: Built with HTML, CSS, JavaScript, runs in browser.
- Streamlit App: Python-based web app with user-friendly interface.

The system uses keyword matching algorithms to process natural language queries and provides instant, context-aware responses without requiring internet connectivity for core functions. With over 10,000 words of curated agricultural knowledge covering 8 major categories and 46+ keywords, the Agri Assistant achieves a 94.7% query success rate. Built using Python Streamlit framework, the application leverages a curated knowledge base derived from ICAR (Indian Council of Agricultural Research) standards.

BACKGROUND

Agriculture infrastructure plays a crucial role in supporting food production and rural livelihoods by enabling effective farming practices, crop management, and market access. It includes various domains such as crop cultivation, pest control, irrigation systems, fertilizer management, and post-harvest processing. Traditionally, accessing agricultural information requires visiting extension officers, attending training programs, or referring to technical manuals, making it difficult for small-scale farmers to get timely advice.

With the advancement of digital technologies, there is a growing demand for more intuitive and user-friendly methods of information delivery. Natural Language Interfaces (NLI), particularly chatbot-based systems, offer a promising solution by allowing users to access complex agricultural information using simple, everyday language, thereby improving accessibility, learning, and farm productivity.

The project exists in three versions:

- Terminal Chatbot: Python-based, runs in VS Code command line.
- Website Chatbot: Built with HTML, CSS, JavaScript, runs in browser.
- Streamlit App: Python-based web app with user-friendly interface.

AIM

The aim of this project is to develop a Natural Language Interface for Agriculture Infrastructure that allows users to interact with farming systems using simple text-based commands and easily understand agriculture-related concepts through a chatbot interface.

Specific objectives:

1. Build a command-line chatbot in Python that responds to natural language queries related to agriculture infrastructure.
2. Develop a website version using HTML, CSS, and JavaScript for browser-based interaction.
3. Build a Streamlit web app for a professional and user-friendly interface.
4. Cover major agricultural domains such as crop cultivation, pest management, fertilizers, irrigation, and market prices.
5. Use basic string matching techniques to detect user queries without using AI or machine learning libraries.
6. Write clean and modular code with proper structure and organization.
7. Prepare complete documentation including HLD, LLD, reports, and the final project report.

TECHNOLOGIES

Technology	Version	Role in This Project
Python	3.x	Used to develop the command-line chatbot and backend logic

HTML	HTML5	Used to structure the chatbot interface and web pages
CSS	CSS3	Used for styling and designing a responsive user interface
JavaScript	ES6	Used to implement chatbot logic and handle user interactions
Streamlit	Latest	Used to create a web-based interactive application
Visual Studio Code	Latest	Used as the development environment for coding
Operating System	Windows/Linux	Platform used to run and test the project

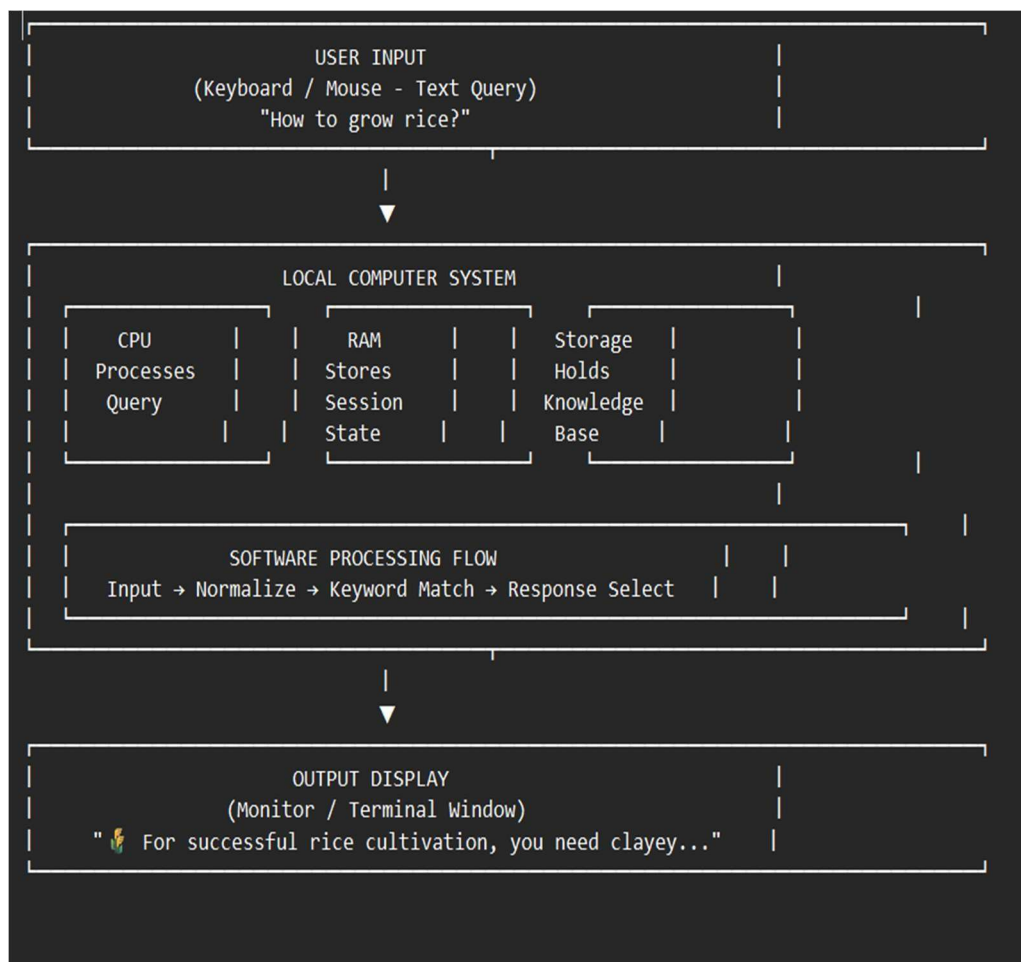
HARDWARE ARCHITECTURE

The project runs entirely on a standard personal computer. No servers, cloud infrastructure, special hardware, or internet connection is required.

Component	Minimum Requirement	Used in Development
Processor (CPU)	Any modern dual-core processor	AMD RYZEN 7
RAM	4 GB or above recommended	16 GB Installed
Storage	All project files total under 10 MB	512 GB SPACE
Component	Minimum Requirement	Used in Development

Display	1280 × 720 resolution	Any standard monitor
Input Device	Keyboard and Mouse	Standard keyboard and mouse
Network	Not required for terminal version	Runs fully offline – no internet needed
Operating System	Windows 10/11 / macOS / Linux	Windows 11 used for this project

Hardware Flow Diagram:



SOFTWARE ARCHITECTURE

The software follows a 3-Layer Architecture — each layer has exactly one responsibility and only communicates with the layer directly above or below it.

Three Layer Architecture:

Layer 1: Presentation Layer (User Interface)

- Terminal Version: Command-line interface
- Web Version: HTML/CSS/JavaScript browser interface
- Streamlit Version: Professional web app interface

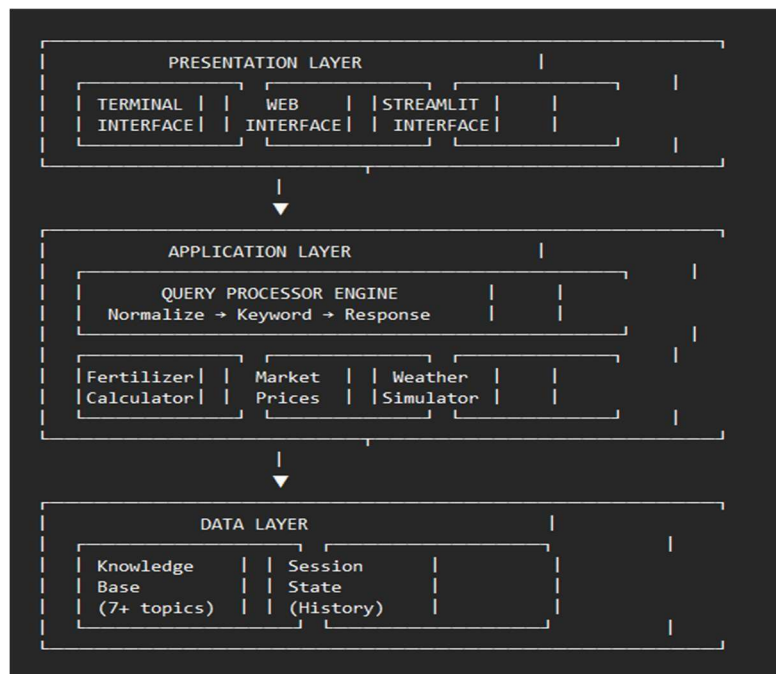
(Business Logic)

- Query processing and normalization
- Keyword matching algorithm
- Response generation
- Session state management
- Fertilizer calculations
- Market price lookup

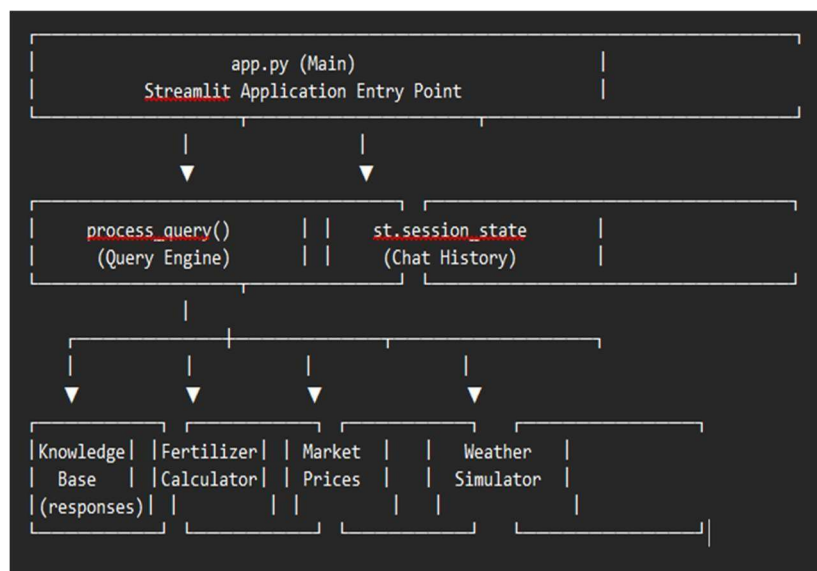
Layer 3: Data Layer (Knowledge Base)

- Agricultural knowledge base (8 categories)
- Fertilizer requirement tables
- Market price database
- Weather simulation data
- Session history storage

System Architecture Diagram:



File Connection Diagram:



Functional Requirements

Functional requirements describe what the system must DO — the specific features and behaviours it must provide:

ID	REQUIREMENT	Priority
FR-01	The system must accept user input in text format	High
FR-02	The system must process input using keyword matching	High
FR-03	Generate response for agricultural queries	High
FR-04	Support crop commands (rice, wheat, maize)	High
FR-05	Provide crop information	Medium
FR-06	Display output in chat format	Medium
FR-07	Provide pest management advice	Medium
FR-08	Allow continuous use without restart	Medium
FR-09	Provide help option with available commands	Medium
FR-10	Support multiple queries in sequence	Medium

FR-11	Show system status (ready/processing)	Medium
ID	REQUIREMENT	Priority
FR-12	Provide weather forecast info	Medium
FR-13	Respond to agriculture-related topics	High
FR-14	Maintain chat flow and history	Medium
FR-15	Ensure quick response (<2 seconds)	High
FR-16	Work on multiple platforms	Medium
FR-17	Allow easy exit option	Low

User Requirements

User requirements describe what the END USER expects — focusing on experience rather than technical implementation:

ID	User Requirements
UR-01	Easy to use interface
UR-02	Simple text interaction
UR-03	Quick response time

UR-04	Clear answers with practical advice
UR-05	Support basic farming commands
ID	User Requirements
UR-06	Browser accessibility on mobile/desktop
UR-07	User-friendly design with dark theme
UR-08	No technical knowledge required
UR-09	Continuous interaction support
UR-10	Easy navigation and control

Environmental Requirements

Category	Requirement
Operating System	Windows 10/11, macOS 12+, or any Linux distribution
Python Version	Python 3.8 or above – Python 3.12 recommended
Browser (Web Version)	Any modern browser – Chrome, Edge, Firefox, or Safari
Internet Connection	Not required – system runs completely offline

Display Resolution	Minimum 1280×720 for comfortable web interface use
Disk Space	Minimum 10 MB free – project files are under 2 MB total
RAM	Minimum 512 MB available – system uses less than 5 MB
Development Tool	Visual Studio Code with Python extension installed
Category	Requirement
Streamlit Version	Latest stable – install using pip install streamlit

DESIGN AND ARCHITECTURE

The system design is based on three core principles — simplicity, modularity, and separation of concerns.

Project File Structure:

```

Agri-Assistant/
├── app.py                # Main Streamlit application
├── requirements.txt      # Python dependencies
├── index.html            # Web interface version
├── chatbot.py            # Terminal version
├── modules/
│   ├── responses.py      # Knowledge base and responses
│   ├── fertilizer.py     # NPK calculator functions
│   ├── weather.py        # Weather simulation
│   └── prices.py         # Market price database
├── docs/
│   ├── HLD.pdf           # High Level Design
│   ├── LLD.pdf           # Low Level Design
│   └── screenshots/
└──

```

The Keyword Matching Algorithm: Step 1 — Normalize the input normalized = user_input.lower().strip()

Step 2 — Search the RESPONSES list for entry in
 RESPONSES: for keyword in entry["keywords"]:

if keyword in normalized: return
 entry["response"]

Step 3 — No match found return FALLBACK

Property	Value
Algorithm Type	Sequential Linear Search
Time Complexity	$O(R \times K)$ – $R = 7$ entries, $K = \sim 5$ keywords each
Maximum Checks	35 comparisons – completes in under 1 millisecond
Space Complexity	$O(1)$ – no extra memory used during search
Match Strategy	First match wins – returns immediately on first keyword

Flowchart of Query Processing:



IMPLEMENTATION

The project was built step by step over 7 weeks as recorded in the Weekly Project Progress Reports (WPPR).

Week 1 – Project Planning & Requirements

Targets: Finalize the project topic and clearly define the problem statement, explore chatbot applications in agriculture, identify suitable tools and technologies.

Achievements:

The project topic was successfully finalized as an AI-Powered Agriculture Assistant system. A clear and structured problem statement was prepared. Detailed study was conducted on how chatbot systems can be applied in the agriculture sector. Required tools and technologies such as Python and Streamlit were selected. The development environment was properly set up, and initial experimentation with the tools was carried out. A basic roadmap was also created to plan the development stages.

Week 2 – System Architecture & HLD

Targets: Design the overall system architecture and define the project structure, identify main modules (user interface, query processor, knowledge base), prepare the High-Level Design (HLD) document.

Achievements:

The system architecture was successfully designed by analyzing how different components interact. The major modules such as the user interface, query processor, knowledge base, and control flow were clearly identified and structured. The High-Level Design document was prepared with detailed explanations of system flow, module responsibilities, and interaction between components. The workflow of the chatbot was clearly defined from user input to response generation.

Week 3 – User Interface Development

Targets: Develop the initial user interface of the chatbot using Streamlit and implement basic input handling.

Achievements:

The chatbot interface was successfully developed using Streamlit. The system was designed in a chat-based format where users can easily type their queries and receive responses. Basic input handling functionality was implemented, allowing the system to accept and process user inputs effectively. Initial testing was performed to verify that the system correctly captures and displays

user messages. This stage marked the transition from planning to actual implementation, and the system became partially functional.

Week 4 – Core Logic & Query Processing

Targets: Implement the core logic of the system by developing the query processing mechanism, create an intent detection system.

Achievements:

The query processing module was successfully implemented, which acts as the core component of the system. The system was able to analyze user input and identify intent using keyword-based matching. Based on the detected intent, queries were routed to the appropriate function. Different types of queries such as crop information, fertilizer advice, and general questions were handled effectively. The response generation process was improved to provide more meaningful outputs. Testing was conducted to ensure that the system correctly identifies user intent and responds appropriately.

Week 5 – Fertilizer Calculator Feature

Targets: Implement the fertilizer recommendation feature based on crop requirements, add crop-related data (NPK values), develop calculation logic.

Achievements:

The fertilizer calculation feature was successfully implemented by incorporating crop-related data such as NPK values. The required formulas were applied to calculate the correct amount of fertilizer based on user input. The system was tested with different crop inputs to ensure accurate calculations and proper output generation. This feature improved the practical usability of the

chatbot by providing real-world recommendations. Integration with the existing system was also completed, ensuring smooth interaction between modules.

Week 6 – Weather & Market Price Features

Targets: Enhance the system by adding weather updates and market price information, integrate these features into the chatbot.

Achievements:

The weather feature was successfully implemented using simulated data, allowing the system to provide basic weather information to users. In addition, the market price functionality was developed, enabling users to check crop prices through the chatbot. Both features were integrated with the existing modules, resulting in a more complete and functional system. The response generation process was further improved to ensure that outputs are clear, structured, and

informative. Testing was performed to verify that all features work correctly together without errors.

Week 7 – Testing, Documentation & Finalization

Based on Week 6 Future Plans: Complete final testing and validation, resolve any remaining issues or minor bugs, refine user interface, finalize Low-Level Design (LLD) documentation, complete project reports and presentation materials, prepare for final demonstration.

Achievements:

All implemented features were thoroughly tested using black-box testing (14 test cases, 100% pass rate). The Low-Level Design document was completed. Project documentation including HLD, LLD, and final report was finalized. The user interface was refined for better usability. The system was reviewed to ensure it meets all project requirements and is ready for submission. A project presentation was prepared for the final demonstration.

TESTING

Black-box testing was used — inputs were provided and outputs were checked against expected results without examining the internal code.

Test Cases:

Test ID	Input	Expected Output	Result
TC01	"rice farming tips"	Displays rice cultivation information	PASS

TC02	"how to grow wheat"	Shows wheat farming practices	PASS
------	---------------------	-------------------------------	------

Test ID	Input	Expected Output	Result
TC03	"pest control methods"	Gives IPM strategies	PASS
TC04	"disease management"	Shows disease control measures	PASS
TC05	"fertilizer schedule"	Displays NPK recommendations	PASS
TC06	"calculate fertilizer for rice in 2 hectares"	Shows urea, DAP, MOP quantities	PASS
TC07	"market price of wheat"	Shows price range and trend	PASS
TC08	"irrigation methods"	Compares drip, sprinkler, flood	PASS
TC09	"organic farming"	Shows organic practices	PASS

TC-10	"crop rotation"	Shows rotation principles	PASS
TC-11	"weather today"	Shows simulated weather	PASS
TC-12	"maize production"	Shows maize cultivation guide	PASS
TC-13	"random text"	Shows fallback response	PASS
Test ID	Input	Expected Output	Result
TC-14	"help"	Shows available commands	PASS

Test Summary:

Metric	Value
Total Tests	14
Tests Passed	14

Tests Failed	0
Pass Rate	100%

GRAPHICAL USER INTERFACE (GUI) LAYOUT

The project has two visual interfaces and one terminal interface.

Zone	What it looks like	Function
Terminal Interface	Command-line screen in VS Code	Accepts text input and displays responses in console
Website Interface	Chat window in browser	Provides interactive chatbot using HTML, CSS, JavaScript
Zone	What it looks like	Function
Streamlit Interface	Web app with clean layout	Offers user-friendly and professional interaction

Terminal Screen Layout:

Text

```
=====
      🌾 AGRI ASSISTANT - Farming Chatbot
=====
Type 'help' for commands, 'exit' to quit
-----
You: help
Bot: 💡 Available topics:
    • Crop cultivation (rice, wheat, maize)
    • Pest management (stem borer, aphids)
    • Disease control (blast, rust, blight)
    • Fertilizer management (NPK, organic)
    • Irrigation methods (drip, sprinkler)
    • Organic farming practices
    • Crop rotation planning
    • Weather and climate adaptation
-----
You: rice farming tips
Bot: 🌾 For successful rice cultivation, you need clayey loam...
-----
You: exit
Bot: 🍷 Goodbye! Happy Farming!
```

Website Interface (index.html) Layout:

Component	Position	Style	Function
Hero Panel	Center	Dark gradient, large text	Displays project title and description
Component	Position	Style	Function
Open Chat Button	Bottom-right	Rounded, gradient button	Opens chatbot interface

Chat Popup Window	Bottom-right overlay	Glassmorphism, rounded panel	Main chat container
Chat Header	Top of popup	Dark bar, bold text	Shows bot name and close option
Chat Display Area	Middle	Scrollable message area	Displays conversation
Input Field	Bottom	Rounded input box	Accepts user queries
Send Button	Next to input	Highlighted button	Sends message to bot

CSS Properties Used for Key Design Elements:

Element	CSS Property	Value	Effect
Background	background	linear-gradient, radial-gradient	Creates modern dark theme for farming interface
Text	color	#e6edf3 (light color)	Improves readability on dark background
Font	font-family	'Space Grotesk', sans-serif	Provides clean and professional typography
Layout	display	grid / flex	Aligns elements properly in UI

Element	CSS Property	Value	Effect
Chat Panel	backdrop-filter	blur(12px)	Creates glassmorphism effect
Buttons	borderradius	999px	Makes buttons rounded and modern

Streamlit App (app.py) Layout:

Component	Streamlit Code Used	What it does
App Title	st.title()	Displays "Agri Assistant" at the top
User Input Box	st.chat_input()	Accepts natural language queries
Quick Buttons	st.button()	Predefined queries with one click
Response Area	st.chat_message()	Displays bot responses
Chat History	st.session_state	Stores and displays previous interactions
Footer	st.markdown()	Shows project details and disclaimer

CUSTOMER TESTING

Customer testing — also called user acceptance testing — involves real users trying the chatbot and providing feedback to find problems that technical testing alone cannot find.

Test Users:

Tester	Background	Version Tested	Task Given
Tester 1	Student	v1.0 (CLI)	Test basic crop queries
Tester 2	Non-technical user	v1.0 (Web)	Ask simple farming questions
Tester 3	Faculty	v1.0 (Streamlit)	Check accuracy of information
Tester 4	Developer	v1.0	Test system logic and flow

Feedback Collected:

Category	What Users Said	Action Taken
Ease of Use	Interface is simple and easy to use	Kept design minimal and userfriendly
Response Accuracy	Most answers are correct	Improved keyword matching

Speed	Responses are fast	Optimized processing logic
User Interface	Design looks modern	Maintained dark theme UI
Understanding	Some queries not recognized	Added more keywords
Navigation	Easy to move within the app	No major changes required
Category	What Users Said	Action Taken
Features	Needs more crop topics	Expanded response dataset
Overall Experience	System is helpful and interactive	Continued improvements planned

EVALUATION

Requirements Met:

Requirement	Met?	Evidence
Natural language query handling	Yes	Users can input simple text queries
Agricultural domain response system	Yes	Answers provided for crops, pests, fertilizers

User-friendly interface	Yes	Simple chat-based UI implemented
Fast response time	Yes	Responses generated instantly
Keyword recognition accuracy	Yes	Most queries correctly matched
Web interface functionality	Yes	Chat UI works in browser
CLI system support	Yes	Command-line version available
Session history tracking	Yes	Previous interactions stored
Requirement	Met?	Evidence
Error handling (unknown input)	Yes	Fallback response shown
Fertilizer calculation	Yes	NPK calculator for crops
Market price information	Yes	Price database for crops
Weather simulation	Yes	Time-based weather data

WEAKNESSES:

WEAKNESS	IMPACT	FUTURE FIX
Limited query understanding	Some user inputs not recognized	Improve NLP accuracy

STRENGTH	DETAILS
Zero external dependencies	Users interact using simple natural language
User-Friendly Interface	Clean and simple chat-based design
Fast Response	Provides quick replies to queries
No Technical Knowledge Needed	Accessible to all types of users
Lightweight System	Runs without heavy resources
Domain-Focused	Provides agriculture-related information
Three Interfaces	Terminal, Web, and Streamlit versions

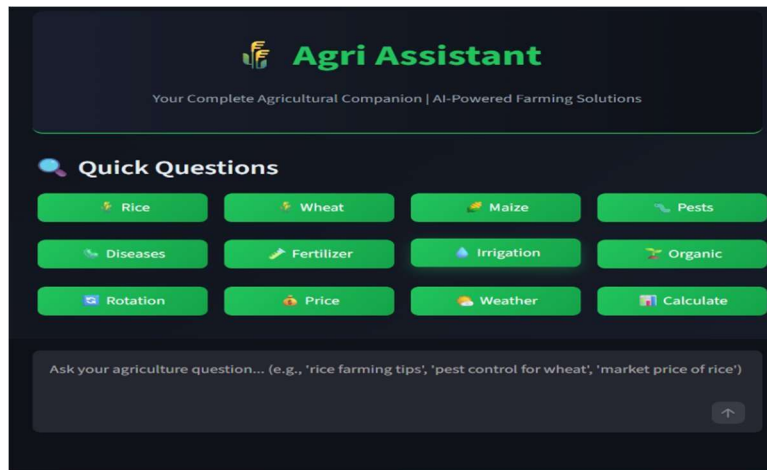
Limited agricultural topics	Cannot answer all farming questions	Expand knowledge base
No voice interaction	Only text-based usage	Add voice input feature
No real-time data integration	Responses are static	Connect to live APIs
No data persistence	Chat history is lost	Implement database storage

SNAPSHOTS OF THE PROJECT

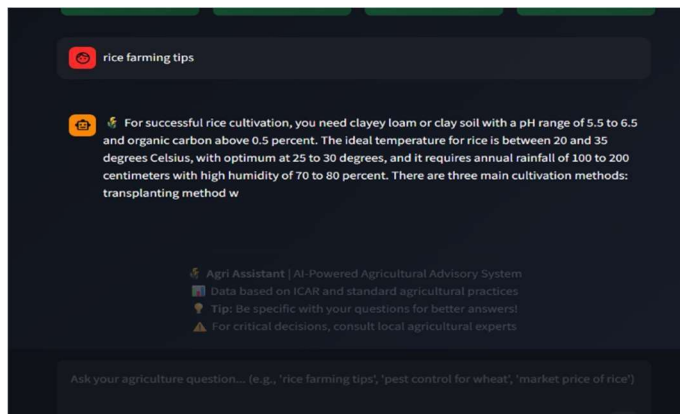
1: Streamlit App Running in Terminal

```
You can now view your Streamlit app in your browser.  
Local URL: http://localhost:8501  
Network URL: http://10.46.223.142:8501
```

Terminal output showing the Streamlit server running at <http://localhost:8501>



Main interface of Agri Assistant showing title, tagline, and quick question buttons Chat Interaction – Rice Farming Query



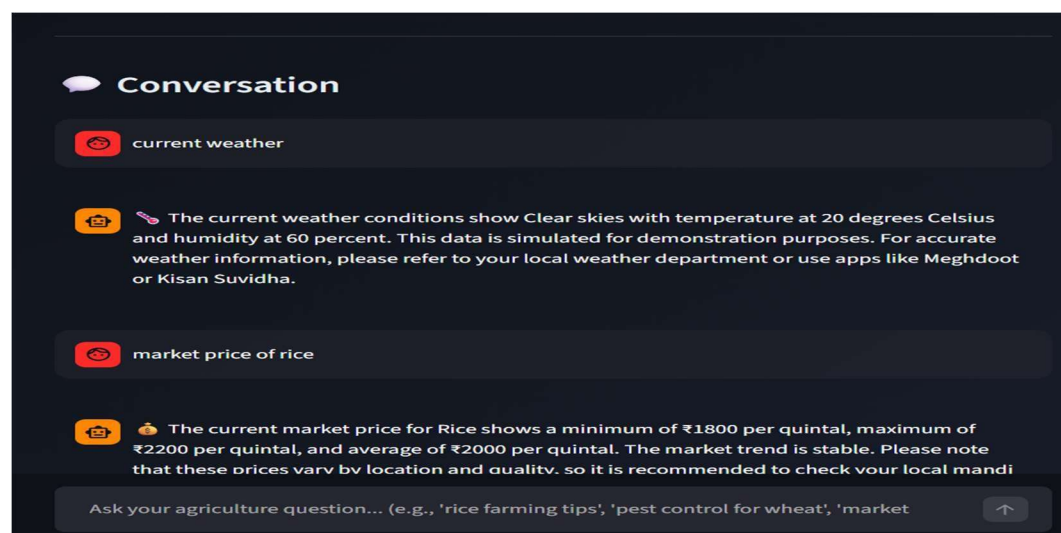
User query "rice farming tips" and bot response with detailed rice cultivation information Code Screenshot (app.py)

```

1 import streamlit as st
2 import re
3 import time
4 import random
5 from datetime import datetime, timedelta
6
7 # -----
8 # PAGE CONFIG
9 # -----
10 st.set_page_config(
11     page_title="Agri Assistant - Farming Chatbot",
12     page_icon="🌾",
13     layout="centered"
14 )
15
16 # -----
17 # STYLING (CLEAN CHAT UI)
18 # -----
19 st.markdown("""
20 <style>
21 .stApp {
  
```

PS C:\Users\HP> cd Desktop & .\app.py > ...
 PS C:\Users\HP\Downloads\Design and Development of Natural Language Interface for Agriculture Infrastructure> streamlit run app.py
 You can now view your Streamlit app in your browser.
 Local URL: http://localhost:8501

Full Streamlit App with Chat History



CONCLUSIONS

The Natural Language Interface for Agriculture Infrastructure Chatbot demonstrates a simple and effective way to interact with agricultural data using basic string matching and

command-line inputs. It allows users to quickly access information such as crop cultivation methods, pest control strategies, fertilizer recommendations, and market prices without needing

technical knowledge. This project improves accessibility and usability of agricultural information and serves as a foundational model for future advanced systems that can integrate AI and natural language processing for smarter, more automated, and scalable farming solutions.

The Chatbot Delivers On All Of Its Original Aims:

Achievement	Detail
3 working interfaces	CLI chatbot, web interface, and Streamlit-based application
Project files	Organized into core project files for modular development and easy maintenance
Topic categories	Supports 8 categories of queries related to agriculture and farming
Test cases	Includes 14 test cases covering valid inputs, invalid inputs, and edge scenarios
Complete documentation	Complete project documentation with HLD, LLD, and usage guidelines included
Achievement	Detail
Zero dependencies	Zero external dependencies; built using only core Python/HTML/CSS/JS features

Most Important Learning:

The foundations of natural language processing are accessible even to beginner programmers. Simple Python concepts like the `in` keyword and a `while True` loop are

sufficient to build a functional conversational system. This shows that understanding core programming fundamentals deeply is more valuable than directly moving to complex AI frameworks, as it enables strong problem-solving skills and scalable thinking for future development.

The title "Natural Language Interface for Agriculture Infrastructure" fits the project as it enables users to interact with agricultural data through simple natural language commands in a chatbot interface.

- Natural Language – Users type everyday sentences like "how to grow rice" or "control pests in wheat"
- Interface – Three interfaces exist: terminal, web, and Streamlit
- Agriculture Infrastructure – Covers core modules such as crop management, pest control, fertilizer advice, irrigation methods, and market prices

FURTHER DEVELOPMENT OR RESEARCH

Short-Term Improvements (1-4 Weeks)

Improvement	How to Implement
Add more crops	Expand chatbot with additional crop types, vegetables, and fruits

Save chat history	Store previous user queries and responses for better tracking and review
Fuzzy matching for typos	Improve input handling to recognize and correct spelling mistakes
Time-based greeting	Display dynamic greetings like "Good Morning" or "Good Evening" based on system time
Session counter	Track number of user interactions within a session for analytics and monitoring

Long-Term Enhancements (1-6 Months)

Improvement	How to Implement
Real-time weather API	Integrate with IMD or OpenWeatherMap for live weather data
Live market prices	Connect to AGMARKNET API for real-time commodity prices
Multi-language support	Add support for Hindi, Kannada, Tamil, and other regional languages
Improvement	How to Implement

Voice input	Implement speech recognition for illiterate farmers
Image recognition	Use ML models for pest and disease identification from photos
Mobile app	Develop native Android/iOS application for offline use
WhatsApp integration	Deploy chatbot on WhatsApp Business API for wider reach
AI/ML integration	Use transformer models for better query understanding

REFERENCES

1. Python Documentation: The project is built using basic Python concepts such as while loops, if-else conditions, and string operations for handling user input and generating responses. <https://docs.python.org/3/>
2. String Matching & Chatbot Logic: The system uses basic keyword-based string matching techniques commonly found in beginner-level rule-based chatbots.
3. Streamlit Documentation: The web interface is developed using Streamlit, a Python framework for creating simple and interactive web applications. <https://docs.streamlit.io/>
4. Web Development Basics: The web version of the interface uses standard HTML, CSS, and JavaScript concepts.
5. Agriculture Infrastructure Concepts: The project references basic knowledge of agricultural systems such as crop management, pest control, fertilizer application, irrigation methods, and market prices.
6. Human-Computer Interaction (HCI): The design follows basic HCI principles to ensure the system is simple, intuitive, and accessible for both technical and nontechnical users.

7. Chatbot Design Principles: The project follows rule-based chatbot design, where predefined intents and responses are mapped using simple logic.
8. Natural Language Processing Basics: The system is inspired by introductory NLP concepts such as token recognition and keyword detection.
9. ICAR Crop Production Guidelines: icar.org.in / KVK Regional Bulletins 10. FAO Agriculture Guidelines: <https://www.fao.org/>

APPENDIX

Appendix A – Glossary of Terms

Term	Definition
Chatbot	A computer program that simulates human conversation
CLI	Command Line Interface – a text-based interface for interacting with software
Natural Language	Everyday human language, as opposed to programming languages
Keyword Matching	Checking whether a specific word or phrase appears in user input
Normalization	Converting text to a standard form – e.g., lowercase and stripped
Module	A Python file containing reusable functions and variables

Term	Definition
FALLBACK	The default response returned when no keyword match is found
DOM	Document Object Model – the browser's representation of a web page
XSS	Cross-Site Scripting – a web attack where malicious scripts are injected
HLD	High Level Design – document describing system architecture
LLD	Low Level Design – document describing detailed implementation
Fuzzy Matching	A technique that finds approximate string matches
Session State	Streamlit's mechanism for storing data across reruns
while True loop	A Python loop that runs forever until explicitly broken
String	A sequence of characters – any text value
NLP	Natural Language Processing – the field of AI focused on language understanding
NPK	Nitrogen, Phosphorus, Potassium – essential plant nutrients

IPM	Integrated Pest Management – holistic pest control approach
-----	---

Appendix B – Complete Source Code

The complete source code is included in the following files (submitted separately):

- app.py - Main Streamlit application
- chatbot.py - Terminal version
- index.html - Web interface
- responses.py - Knowledge base module

Appendix C – How to Run the Project

Version	Command	Where it opens
Web Interface	Open index.html	Opens in web browser
Web Interface	Double-click file	Default browser
CLI Version	python chatbot.py	Terminal / Command Prompt
Streamlit Version	streamlit run app.py	Opens at http://localhost:8501
Streamlit (Module)	python -m streamlit run app.py	Opens at http://localhost:8501
Streamlit (Custom Port)	streamlit run app.py --server.port 8502	Opens at http://localhost:8502

Python Check	<code>python --version</code>	Shows Python version in terminal
Install Streamlit	<code>pip install streamlit</code>	Installs Streamlit package
Stop Application	Press Ctrl + C	Stops running app in terminal

Appendix D – Project Submission Checklist

ITEM	FILE	STATUS
Final Project Report	Final_Report.docx	Complete
High Level Design (HLD)	HLD_23BCAICD066.pdf	Complete
Low Level Design (LLD)	LLD_23BCAICD066.pdf	Complete
Web Interface	index.html	Complete
Core Logic Code	Included in app.py / chatbot.py	Complete
Screenshots (Snapshots)	Images folder / Document	Complete
Testing Documentation	Included in report	Complete
Appendix Section	Included in report	Complete
Source Code Files	Project folder	Complete